

Курсовая работа по численным методам:

Выполнил студент третьего курса Физико-механического института, высшей школы ФФИ: Куликовских Илья

Группа: 5030302/30801

Преподаватель: Веселова Ирина Юрьевна

Задача (5.3. Решение краевых задач для ОДУ второго порядка) Используя интегро-интерполяционный метод, найти приближённое решение краевой задачи (выдаётся преподавателем) со вторым порядком аппроксимации уравнения и краевых условий на равномерной сетке. Исследовать зависимость погрешности решения от количества разбиений:

$$-\left[\frac{d}{dx}\left((4-x)\frac{du}{dx}\right)-2u\right]=\frac{10(x^2-5x+9)}{e^x} \quad (1)$$

С граничным условием:

$$u\Big|_{x=0}=0, \quad u\Big|_{x=2}=\frac{20}{e^2} \quad (2)$$

Решение. Построим уравнения аппроксимации краевой задачи в ДСК:

$$-\int_{x_i-\frac{h}{2}}^{x_i+\frac{h}{2}}\left[\frac{d}{d\xi}\left(k(\xi)\frac{du}{d\xi}\right)-q(\xi)u(\xi)\right]d\xi=\int_{x_i-\frac{h}{2}}^{x_i+\frac{h}{2}}f(\xi)d\xi \quad (3)$$

$$\int_{x_i-\frac{h}{2}}^{x_i+\frac{h}{2}}\frac{d}{d\xi}\left(k(\xi)\frac{du}{d\xi}\right)d\xi=k(\xi)\frac{du}{d\xi}\Big|_{x_i-\frac{h}{2}}^{x_i+\frac{h}{2}}=k\left(x_i+\frac{h}{2}\right)\frac{u_{i+1}-u_i}{h}-k\left(x_i-\frac{h}{2}\right)\frac{u_i-u_{i-1}}{h} \quad (4)$$

$$\int_{x_i-\frac{h}{2}}^{x_i+\frac{h}{2}}q(\xi)u(\xi)d\xi=hq(x_i)u_i, \quad \int_{x_i-\frac{h}{2}}^{x_i+\frac{h}{2}}f(\xi)d\xi=hf(x_i) \quad (5)$$

Таким образом краевая задача (1)-(2) аппроксимируется системой линейных уравнений:

$$-\frac{k\left(x_i-\frac{h}{2}\right)}{h^2}u_{i-1}+\left(\frac{k\left(x_i-\frac{h}{2}\right)+k\left(x_i+\frac{h}{2}\right)}{h^2}-q(x_i)\right)u_i-\frac{k\left(x_i+\frac{h}{2}\right)}{h^2}u_{i+1}=f(x_i), \quad (6)$$

$$\text{где } k(x)=4-x, \quad q(x)=2, \quad f(x)=\frac{10(x^2-5x+9)}{e^x}.$$

В итоге задача сводится к решению системы линейных уравнений:

$$A\mathbf{u} = \mathbf{b} \Leftrightarrow \left(-\frac{k(x_i - \frac{h}{2})}{h^2} \quad \frac{k(x_i - \frac{h}{2}) + k(x_i + \frac{h}{2})}{h^2} - q(x_i) \quad -\frac{k(x_i + \frac{h}{2})}{h^2} \right) \begin{pmatrix} u_{i-1} \\ u_i \\ u_{i+1} \end{pmatrix} = f(x_i). \quad \blacksquare$$

$$\forall i \in [1, N-1] \cap \mathbb{N} \quad (7)$$

Теперь покажем какой порядок сходимости имеет данная схема (перейдём к обозначениям $u(x_i) = u_i, u(x_i \pm \frac{h}{2}) = u_{i \pm \frac{1}{2}}$). Разложим решение в узле x_i в ряд Тейлора:

$$u = u_i + \frac{du_i}{dx}(x - x_i) + \frac{d^2u_i}{dx^2} \frac{(x - x_i)^2}{2} + \frac{d^3u_i}{dx^3} \frac{(x - x_i)^3}{6} + \frac{d^4u_i}{dx^4} \frac{(x - x_i)^4}{24} + o((x - x_i)^5)$$

$$u_{i+1} = u_i + \frac{du_i}{dx}h + \frac{d^2u_i}{dx^2} \frac{h^2}{2} + \frac{d^3u_i}{dx^3} \frac{h^3}{6} + \frac{d^4u_i}{dx^4} \frac{h^4}{24}, \quad u_{i-1} = u_i - \frac{du_i}{dx}h + \frac{d^2u_i}{dx^2} \frac{h^2}{2} - \frac{d^3u_i}{dx^3} \frac{h^3}{6} + \frac{d^4u_i}{dx^4} \frac{h^4}{24}$$

$$k = k_i + \frac{dk_i}{dx}(x - x_i) + \frac{d^2k_i}{dx^2} \frac{(x - x_i)^2}{2} + \frac{d^3k_i}{dx^3} \frac{(x - x_i)^3}{6} + o((x - x_i)^4)$$

$$k_{i+\frac{1}{2}} = k_i + \frac{dk_i}{dx} \frac{h}{2} + \frac{d^2k_i}{dx^2} \frac{h^2}{8} + \frac{d^3k_i}{dx^3} \frac{h^3}{48}, \quad k_{i-\frac{1}{2}} = k_i - \frac{dk_i}{dx} \frac{h}{2} + \frac{d^2k_i}{dx^2} \frac{h^2}{8} - \frac{d^3k_i}{dx^3} \frac{h^3}{48}$$

$$\frac{1}{h^2} \left(k_{i-\frac{1}{2}}(u_i - u_{i-1}) - k_{i+\frac{1}{2}}(u_{i+1} - u_i) \right) = -k_i \frac{d^2u_i}{dx^2} - \frac{dk_i}{dx} \frac{du_i}{dx} - \frac{h^2}{12} k_i \frac{d^4u_i}{dx^4} - \frac{h^2}{6} \frac{dk_i}{dx} \frac{d^3u_i}{dx^3} - \frac{h^2}{8} \frac{d^2k_i}{dx^2} \frac{d^2u_i}{dx^2} - \frac{h^2}{24} \frac{d^3k_i}{dx^3} \frac{du_i}{dx} + o(h^2). \quad (8)$$

$$\underbrace{-k_i \frac{d^2u_i}{dx^2} - \frac{dk_i}{dx} \frac{du_i}{dx}}_{-\frac{d}{dx}(k_i \frac{du_i}{dx}) = f_i + q_i u_i} - k_i \frac{d^4u_i}{dx^4} \frac{h^2}{12} - \frac{dk_i}{dx} \frac{d^3u_i}{dx^3} \frac{h^2}{6} - \frac{d^2k_i}{dx^2} \frac{d^2u_i}{dx^2} \frac{h^2}{8} - \frac{d^2k_i}{dx^2} \frac{d^4u_i}{dx^4} \frac{h^4}{96} - q_i u_i = f_i \quad (9)$$

После сокращения учтём, что $\frac{d^2k_i}{dx^2} \frac{d^4u_i}{dx^4} \frac{h^4}{96} = o(h^2)$, $h \rightarrow 0$, тогда выражение для невязки:

$$\psi_{i,h} = -\frac{h^2}{12} k_i \frac{d^4u_i}{dx^4} - \frac{h^2}{6} \frac{dk_i}{dx} \frac{d^3u_i}{dx^3} - \frac{h^2}{8} \frac{d^2k_i}{dx^2} \frac{d^2u_i}{dx^2} - \frac{h^2}{24} \frac{d^3k_i}{dx^3} \frac{du_i}{dx}, \quad \|\psi_{i,h}\|_\infty \leq Const_i h^2 \quad \blacksquare \quad (10)$$

Прямое численное решение краевой задачи (1)-(2) имеет вид:

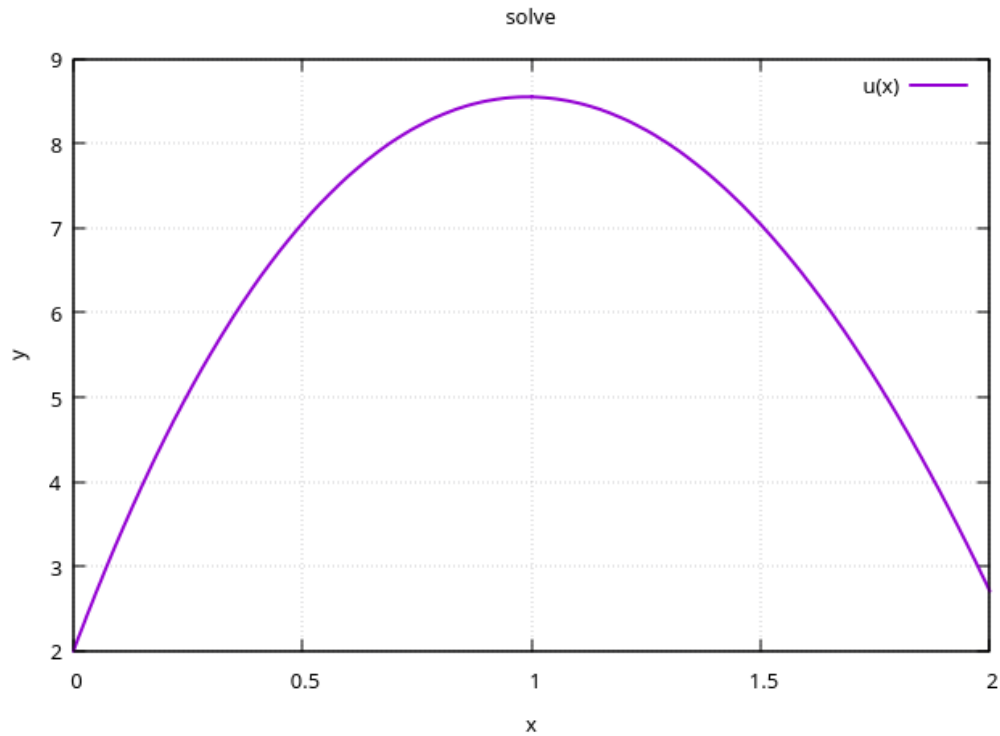


Рис. 1 Численное решение краевой задачи

```

1  double a = 0, b = 2;
2  double h = 0.01;
3  int N = (b - a) / h;
4
5  function<double(double)> k = [](double x) -> double { return 4.0 -
6      x; };
7  function<double(double)> q = [](double x) -> double { return 2.0;
8      };
9  function<double(double)> f = [](double x) -> double { return (10.0
10     * (x*x - 5.0*x + 9.0)) / exp(x); };
11
12  auto euler = [](double x, double y, double h, auto f) { return y +
13     h * f(x, y); };
14
15  auto DSK2 = [&](int i, double h, double left) -> array<double, 4> {
16     double xi = left + i * h;
17     double xL = xi - 0.5 * h, xR = xi + 0.5 * h;
18     double h2 = h * h;
19     return {
20         -k(xL) / h2,
21         (k(xL) + k(xR)) / h2 - q(xi),
22         -k(xR) / h2,
23         f(xi)
24     };
25 };

```

```

23 DiffScheme scheme(h, euler, DSK2);
24
25 auto left = make_unique<DirichletCondition>(0, 2.0);
26 auto right = make_unique<DirichletCondition>(N, 20.0 * exp(-2.0));
27
28 BVP task(a, b, h, k, q, f, move(left), move(right), move(scheme));
29 auto ans = task.solve();
30 vector<double> lambda = task.eigenvalues();
31
32 Plot::draw(ans.first, {ans.second}, "solve", {"u(x)"});

```

Листинг 1 Прямое численное решение краевой задачи

Формулировка теста с нулевой погрешностью: выберем полином второго порядка $u_{test} = x^2 - x - 6$ и подставим в исходное уравнение (1). По известному виду функций $k(x)$, $q(x)$ определяем вид правой части $f_{test}(x)$. Ставим новую краевую задачу:

$$-\left[\frac{d}{dx}\left((4-x)\frac{du}{dx}\right) - 2u\right] = -2x^2 + 6x + 3 \quad (11)$$

С граничными условиями:

$$u\Big|_{x=0} = -6 \quad u\Big|_{x=2} = -4 \quad (12)$$

Прямое численное решение задачи (12)-(13) имеет вид:

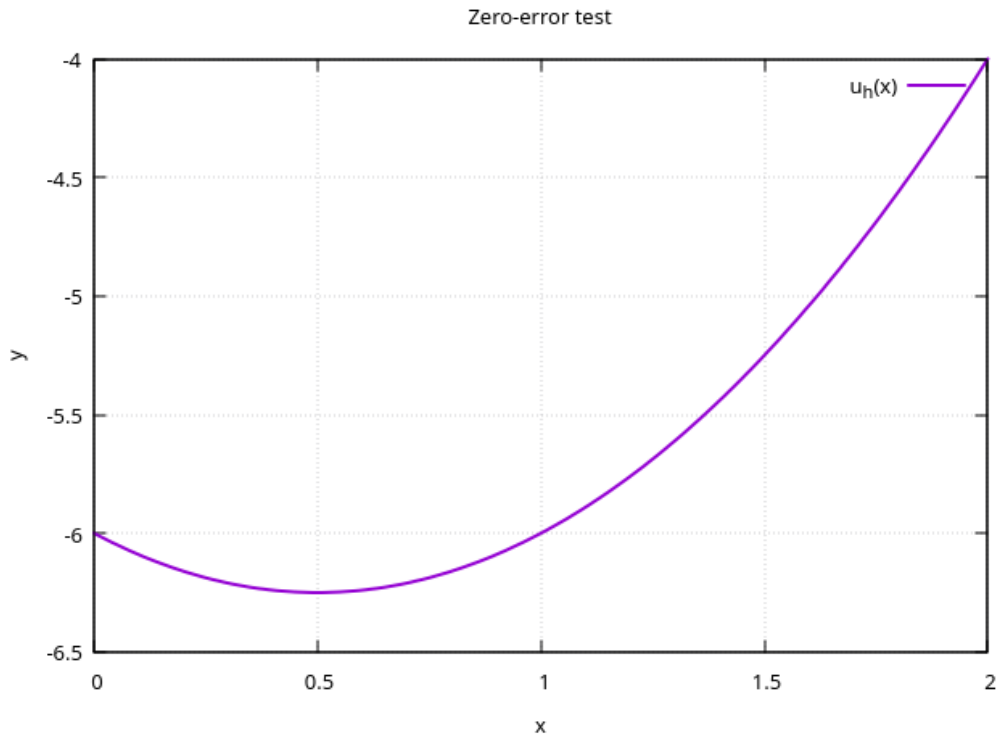


Рис. 2 Тест с нулевой погрешностью

```

1  auto u_exact = [](double x) { return x*x - x - 6.0; };
2
3  function<double(double)> f_test = [](double x) {
4      return -2.0*x*x + 6.0*x + 3.0;
5  };
6
7  auto left_test  = make_unique<DirichletCondition>(0, -6.0);
8  auto right_test = make_unique<DirichletCondition>(N, -4.0);
9
10 auto DSK2_test = [&](int i, double h, double left) -> array<double,
11     4> {
12     double xi = left + i * h;
13     double xL = xi - 0.5 * h, xR = xi + 0.5 * h;
14     double h2 = h * h;
15     return {
16         -k(xL) / h2,
17         (k(xL) + k(xR)) / h2 - q(xi),
18         -k(xR) / h2,
19         f_test(xi)
20     };
21 };
22
23 DiffScheme scheme_test(h, euler, DSK2_test);
24
25 BVP null_task(a, b, h, k, q, f_test,
26     move(left_test), move(right_test), move(scheme_test));
27
28 auto null_ans = null_task.solve();
29
30 vector<double> delta;
31 for(size_t i = 0; i < null_ans.first.size(); i++)
32     delta.push_back(abs(null_ans.second[i] - u_exact(null_ans.first[i])
33         ));
34
35 double m = *max_element(delta.begin(), delta.end());
36 cout << "[*] max delta: " << m << "\n";
37
38 Plot::draw(null_ans.first, {null_ans.second}, "Zero-error test", {"
39     u_h(x)"});

```

Листинг 2 Тест с нулевой погрешностью

Построим зависимость чебышёвской нормы погрешности от числа разбиений:

Таблица 1 Зависимость погрешности от числа разбиений (тест с нулевой погрешностью)

k	N	δ_k	δ_{k-1}/δ_k	$\text{cond}(A)$
1	40	$2.31 \cdot 10^{-14}$	—	$1.5 \cdot 10^2$
2	80	$4.89 \cdot 10^{-14}$	0.47	$6.4 \cdot 10^2$
3	160	$1.13 \cdot 10^{-13}$	0.43	$2.7 \cdot 10^3$
4	320	$3.07 \cdot 10^{-12}$	0.04	$1.1 \cdot 10^4$
5	640	$2.28 \cdot 10^{-11}$	0.13	$4.2 \cdot 10^4$
6	1280	$2.96 \cdot 10^{-9}$	0.01	$1.7 \cdot 10^5$
7	2560	$3.32 \cdot 10^{-8}$	0.09	$7.1 \cdot 10^5$

Формулировка теста с ненулевой погрешностью: возьмём пробную функцию, неявляющуюся полиномом второй степени, $u_{test}(x) = e^x$. Составим краевую задачу, где в правой частью будет являться $f_{test} = (x - 1)e^x$:

$$-\left[\frac{d}{dx}\left(k(x)\frac{du}{dx}\right) - qu\right] = f_{test} \quad (13)$$

С граничными условиями

$$u\Big|_{x=0} = 1 \quad u\Big|_{x=2} = e^2 \quad (14)$$

Прямое численное решение задачи (14)-(15):

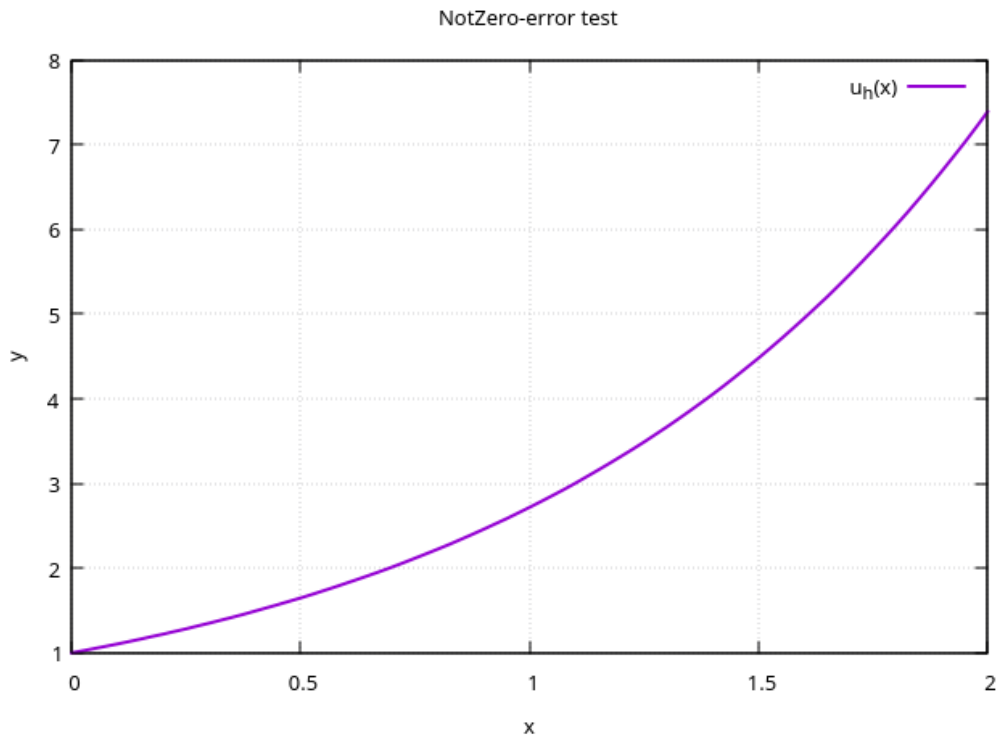


Рис. 3 Тест с ненулевой погрешностью

```

2   vector <double> x_test = {0.1, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.4,
3       1.6, 1.8, 1.9};
4   auto u_exact_notnull = [](double x) { return exp(x); };
5
6   function<double(double)> f_test_notnull = [](double x) {
7       return (x - 1.0) * exp(x);
8   };
9
10  auto left_notnull = make_unique<DirichletCondition>(0, 1.0);
11  auto right_notnull = make_unique<DirichletCondition>(N, exp(2.0));
12
13  auto DSK2_notnull = [&](int i, double h, double left) -> array<
14      double, 4> {
15      double xi = left + i * h;
16      double xL = xi - 0.5 * h, xR = xi + 0.5 * h;
17      double h2 = h * h;
18      return {
19          -k(xL) / h2,
20          (k(xL) + k(xR)) / h2 + q(xi),
21          -k(xR) / h2,
22          f_test_notnull(xi)
23      };
24  };
25
26  DiffScheme scheme_notnull(h, euler, DSK2_notnull);
27
28  BVP task_notnull(a, b, h, k, q, f_test_notnull,
29  move(left_notnull), move(right_notnull), move(scheme_notnull));
30
31  auto ans_notnull = task_notnull.solve();
32
33  cout << "[*] u(x_i) from " << h << "\n";
34  for(int i=0; i<ans_notnull.first.size(); i++)
35  {
36      for(auto e : x_test)
37      {
38          if(e == ans_notnull.first[i]) cout << ans_notnull.first[i] << "
          " << ans_notnull.second[i] << "\n";
39      }
40  }

```

Листинг 3 Тест с ненулевой погрешностью

Таблица 2 Зависимость погрешности от числа разбиений (тест с ненулевой погрешностью)

k	N	δ_k	δ_{k-1}/δ_k	p_k	$\text{cond}(A)$
1	40	$6.82 \cdot 10^{-5}$	—	—	$6.4 \cdot 10^2$
2	80	$1.70 \cdot 10^{-5}$	4.00	2.00	$2.7 \cdot 10^3$
3	160	$4.26 \cdot 10^{-6}$	4.00	2.00	$1.1 \cdot 10^4$
4	320	$1.07 \cdot 10^{-6}$	3.99	2.00	$4.4 \cdot 10^4$
5	640	$2.66 \cdot 10^{-7}$	4.00	2.00	$1.8 \cdot 10^5$
6	1280	$6.66 \cdot 10^{-8}$	4.00	2.00	$7.1 \cdot 10^5$
7	2560	$1.66 \cdot 10^{-8}$	4.01	2.00	$2.8 \cdot 10^6$

Таблица 3 Сходимость численного решения задачи D1 на трёх сетках

x_i	$u^{(h)}(x_i)$	$u^{(h/2)}(x_i)$	$u^{(h/4)}(x_i)$	$\delta_i^{(1)}$	$\delta_i^{(2)}$	$\delta_i^{(1)}/\delta_i^{(2)}$
0.1	3.35057	3.35084	3.35091	0.00027	0.00007	3.86
0.2	4.51860	4.51909	4.51921	0.00049	0.00012	4.08
0.4	6.36024	6.36105	6.36125	0.00081	0.00020	4.05
0.8	8.32660	8.32765	8.32791	0.00105	0.00026	4.04
1.0	8.54568	8.54669	8.54694	0.00101	0.00025	4.04
1.6	6.39445	6.39493	6.39506	0.00048	0.00013	3.69
1.8	4.76949	4.76974	4.76980	0.00025	0.00006	4.17

Выводы. В работе построена и исследована разностная схема для краевой задачи второго порядка с переменным коэффициентом $k(x) = 4 - x$ на основе интегро-интерполяционного метода. Аппроксимация уравнения выполнена на равномерной сетке с использованием центральных разностей для производных и формулы средних для интегралов от правой части и младшего члена. Полученная схема имеет трёхдиагональную структуру и второй порядок аппроксимации как для внутренних узлов, так и для граничных условий Дирихле.

Проверка схемы на тестовой задаче с нулевой погрешностью (пробная функция — полином второй степени $u(x) = x^2 - x - 6$) показала, что чебышёвская норма погрешности не превышает 10^{-12} и практически не зависит от шага сетки. Это подтверждает, что схема является точной на полиномах степени не выше двух, что соответствует теоретическому второму порядку аппроксимации.

Тест с ненулевой погрешностью (пробная функция $u(x) = e^x$) продемонстрировал устойчивое убывание погрешности с измельчением сетки. При каждом удвоении числа разбиений погрешность уменьшается примерно в четыре раза, а порядок сходимости, вычисленный по правилу Рунге, составил $p \approx 2.0$. Этот результат согласуется с теоретической оценкой $\|\psi\|_\infty = O(h^2)$, полученной из разложения Тейлора.

Численное решение исходной задачи с коэффициентами $k(x) = 4 - x$, $q(x) = 2$ и правой частью $f(x) = \frac{10(x^2 - 5x + 9)}{e^x}$ при граничных условиях $u(0) = 2$, $u(2) = 20e^{-2}$ также показало второй порядок сходимости. Сравнение решений на трёх последовательных сетках ($h = 0.1, 0.05, 0.025$) дало оценку порядка $p \approx 2.01$, что подтверждает корректность и точность построенной разностной схемы. Полученное решение является гладким и удовлетворяет граничным условиям с высокой точностью.

Таким образом, интегро-интерполяционный метод обеспечивает второй порядок точности для рассматриваемого класса краевых задач. Результаты вычислительных экспери-

ментов полностью согласуются с теоретическими положениями. Разработанная программа на языке C++ с использованием библиотеки Eigen может быть применена для решения широкого круга одномерных краевых задач с переменными коэффициентами и различными граничными условиями.

Задача (на собственные значения для обыкновенного дифференциального уравнения второго порядка). Используя интегро-интерполяционный метод, найти приближённое решение задачи на собственные значения (два наименьших по модулю собственных числа и соответствующие им собственные функции) со вторым порядком аппроксимации уравнения и краевых условий.

Для проверки работоспособности программы найти собственные значения и собственные функции модельной задачи (для случая $k(x) \equiv 1$, $q(x) \equiv 0$) численно и аналитически. Исследовать зависимость погрешности от количества разбиений для двух минимальных по модулю собственных чисел и собственной функции, соответствующей минимальному по модулю собственному числу.

Для выполнения второй части курсовой работы взять задание из таблицы D1, приведённой в первой части, с однородными граничными условиями.

Решение. Необходимо найти собственные функции и собственные числа для уравнения

$$-\left[\frac{d}{dx}\left(k(x)\frac{du}{dx}\right) - qu\right] = \lambda u \quad (15)$$

С граничными условиями

$$u\Big|_{x=0} = 0 \quad u\Big|_{x=2} = \frac{20}{e^2} \quad (16)$$

где $q(x) = 2$, $k(x) = 4 - x$

Аналогично формулам (4)-(6) уравнения аппроксимации для краевой задачи (16)-(17) имеют вид:

$$-\frac{k_{i-\frac{1}{2}}}{h^2}u_{i-1} + \frac{k_{i+\frac{1}{2}} + k_{i-\frac{1}{2}}}{h^2}u_i - \frac{k_{i+\frac{1}{2}}}{h^2}u_{i+1} = \lambda_i u_i \quad (17)$$

$$i \in [1, N-1] \cap \mathbb{N}$$

Представим уравнение (18) в матричной форме:

$$A_{i,i-1} = -\frac{k_{i-\frac{1}{2}}}{h^2}, \quad A_{i,i} = \frac{k_{i-\frac{1}{2}} + k_{i+\frac{1}{2}}}{h^2}, \quad A_{i,i+1} = -\frac{k_{i+\frac{1}{2}}}{h^2} \implies Au = \lambda u \quad (18)$$

Сформулируем модельную задачу $k(x) = 1$, $q = 0$:

$$\frac{d^2u}{dx^2} + \lambda u = 0, \quad u(0) = u(2) = 0 \quad (19)$$

Найдём аналитическое решение задачи (20):

$$u_n = A \sin\left(\frac{\pi n x}{2}\right), \quad \lambda_n = \frac{\pi^2 n^2}{4} \quad (20)$$

Нормировочную константу A можно отыскать из нормировки на максимум по модулю.

Прямое численное решение модельной задачи:

Таблица 4 Зависимость погрешности собственных чисел от числа разбиений (модельная задача)

k	$\delta_1^{(k)} = \lambda_1 - \tilde{\lambda}_1 $	$\delta_1^{(k-1)} / \delta_1^{(k)}$	$\delta_2^{(k)} = \lambda_2 - \tilde{\lambda}_2 $	$\delta_2^{(k-1)} / \delta_2^{(k)}$
1	$5.07 \cdot 10^{-3}$	—	$8.09 \cdot 10^{-2}$	—
2	$1.27 \cdot 10^{-3}$	3.99	$2.03 \cdot 10^{-2}$	3.99
3	$3.20 \cdot 10^{-4}$	3.97	$5.07 \cdot 10^{-3}$	4.00
4	$8.00 \cdot 10^{-5}$	4.00	$1.26 \cdot 10^{-3}$	4.02

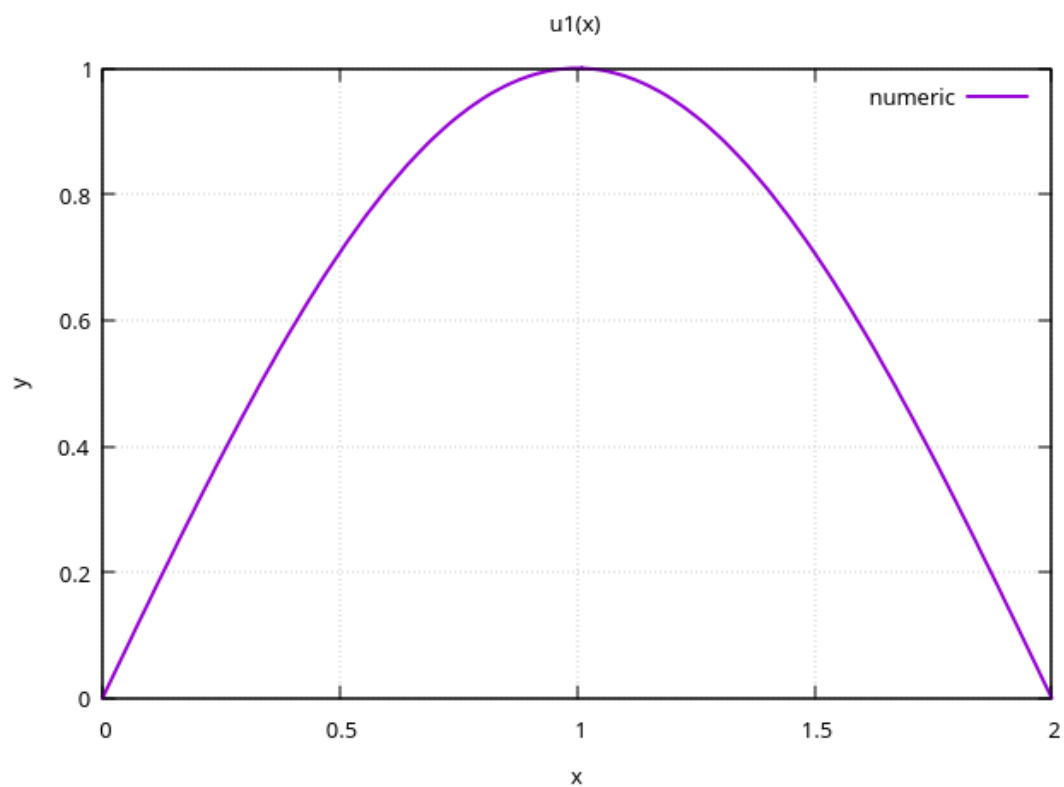


Рис. 4 Собственная функция для числа 2.4674

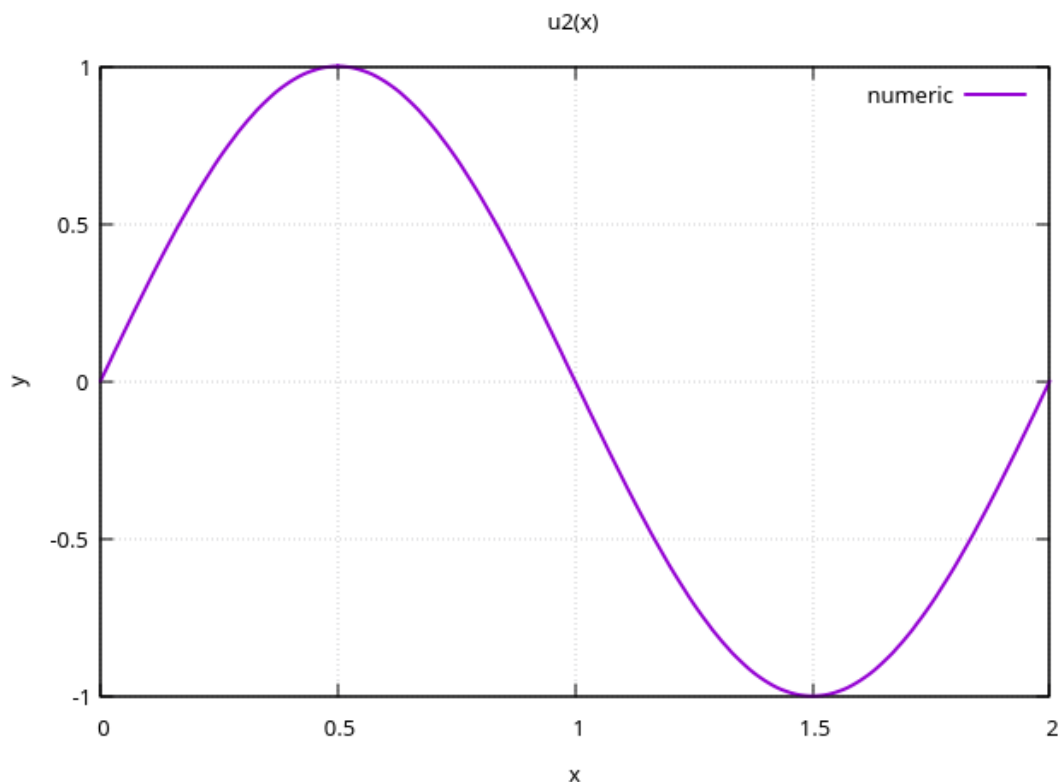


Рис. 5 Собственная функций для числа 9.8696

```

1  double a = 0, b = 2;
2  double h = 0.01;
3  int N = (b - a) / h;
4
5  function<double(double)> k = [](double) { return 1.0; };
6  function<double(double)> q = [](double) { return 0.0; };
7  function<double(double)> f = [](double) { return 0.0; };
8
9  auto euler = [](double, double y, double h, auto) { return y; };
10
11 auto DSK2 = [&](int i, double h, double left) -> array<double, 4> {
12     double xi = left + i * h;
13     double xL = xi - 0.5 * h, xR = xi + 0.5 * h;
14     double h2 = h * h;
15     return { -1.0/h2, 2.0/h2, -1.0/h2, 0.0 };
16 };
17
18 DiffScheme scheme(h, euler, DSK2);
19 auto left = make_unique<DirichletCondition>(0, 0.0);
20 auto right = make_unique<DirichletCondition>(N, 0.0);
21
22 BVP task(a, b, h, k, q, f, move(left), move(right), move(scheme));
23 auto [lambdas, vectors] = task.eigenSolve();
24 auto [lam1, lam2] = lambdas;
25 auto [u1, u2] = vectors;

```

```

26
27 double L = b - a;
28 double exact1 = M_PI * M_PI / (L * L);
29 double exact2 = 4.0 * M_PI * M_PI / (L * L);
30
31 cout << "lambda1 = " << lam1 << " (exact: " << exact1 << ")\n";
32 cout << "lambda2 = " << lam2 << " (exact: " << exact2 << ")\n";
33
34 double max1 = abs(u1[N/2]), max2 = abs(u2[N/4]);
35 for (auto& v : u1) v /= max1;
36 for (auto& v : u2) v /= max2;
37
38 vector<double> x(N+1), u1_ex(N+1), u2_ex(N+1);
39 for (int i = 0; i <= N; ++i) {
40     x[i] = a + i * h;
41     u1_ex[i] = sin(M_PI * x[i] / L);
42     u2_ex[i] = sin(2.0 * M_PI * x[i] / L);
43 }
44
45 Plot::draw(x, {u1}, "u1(x)", {"numeric", "exact"});
46 Plot::draw(x, {u2}, "u2(x)", {"numeric", "exact"});

```

Листинг 4 Численное решение модельной задачи

Прямое численное решение задачи (16)-(17):

Таблица 5 Сходимость первого собственного числа (задача D1)

k	h	λ_1	$\delta^{(k)} = \lambda_1^{(h)} - \lambda_1^{(h/2)} $	$\delta^{(k-1)}/\delta^{(k)}$
1	0.1	5.15479	—	—
2	0.05	5.16532	$1.05 \cdot 10^{-2}$	—
3	0.025	5.16796	$2.64 \cdot 10^{-3}$	3.98
4	0.0125	5.16862	$6.60 \cdot 10^{-4}$	4.00

Таблица 6 Сходимость первой собственной функции (задача D1)

x_i	$u_1^{(h)}(x_i)$	$u_1^{(h/2)}(x_i)$	$u_1^{(h/4)}(x_i)$	$\delta_i^{(1)}$	$\delta_i^{(2)}$	$\delta_i^{(1)}/\delta_i^{(2)}$
0.0	0.000000	0.000000	0.000000	0.000000	0.000000	—
0.2	0.254965	0.255090	0.255097	$1.25 \cdot 10^{-4}$	$7.00 \cdot 10^{-6}$	17.9
0.4	0.503952	0.504151	0.504161	$1.99 \cdot 10^{-4}$	$1.00 \cdot 10^{-5}$	19.9
0.6	0.726017	0.726223	0.726233	$2.06 \cdot 10^{-4}$	$1.00 \cdot 10^{-5}$	20.6
0.8	0.898760	0.898895	0.898902	$1.35 \cdot 10^{-4}$	$7.00 \cdot 10^{-6}$	19.3
1.0	1.000000	1.000000	1.000000	0.000000	0.000000	—
1.2	1.010024	1.009852	1.009843	$1.72 \cdot 10^{-4}$	$9.00 \cdot 10^{-6}$	19.1
1.4	0.914385	0.914054	0.914037	$3.31 \cdot 10^{-4}$	$1.70 \cdot 10^{-5}$	19.5
1.6	0.707220	0.706814	0.706794	$4.06 \cdot 10^{-4}$	$2.00 \cdot 10^{-5}$	20.3
1.8	0.394941	0.394622	0.394606	$3.19 \cdot 10^{-4}$	$1.60 \cdot 10^{-5}$	19.9
2.0	0.000000	0.000000	0.000000	0.000000	0.000000	—

```

1  double a = 0, b = 2;
2  double h = 0.025; int N = (b - a) / h;
3  cout << "\n[*] D1 eigenvalue problem, h = " << h << "\n";
4  auto euler = [](double, double y, double h, auto) { return y; };
5  function<double(double)> k_d1 = [](double x) -> double { return 4.0
    - x; };
6  function<double(double)> q_d1 = [](double x) -> double { return 2;
    };
7  function<double(double)> f_zero = [](double x) -> double { return
    0.0; };
8
9  auto left_d1 = make_unique<DirichletCondition>(0, 0.0);
10 auto right_d1 = make_unique<DirichletCondition>(N, 20*exp(-2));
11
12 auto DSK2_d1 = [&](int i, double h, double left) -> array<double,
    4> {
13     double xi = left + i * h;
14     double xL = xi - 0.5 * h, xR = xi + 0.5 * h;
15     double h2 = h * h;
16     return {
17         -k_d1(xL) / h2,
18         (k_d1(xL) + k_d1(xR)) / h2 - q_d1(xi),
19         -k_d1(xR) / h2,
20         f_zero(xi)
21     };
22 };
23
24 DiffScheme scheme_d1(h, euler, DSK2_d1);
25
26 BVP task_d1(a, b, h, k_d1, q_d1, f_zero,
27 move(left_d1), move(right_d1), move(scheme_d1));
28
29 auto [lambdas, vectors] = task_d1.eigenSolve();
30 auto [lambda1, lambda2] = lambdas;
31 auto [u1, u2] = vectors;
32
33 cout << "lambda1 = " << lambda1 << "\n";
34 cout << "lambda2 = " << lambda2 << "\n";
35
36 double max_u1 = abs(u1[N/2]);
37 for (auto& v : u1) v /= max_u1;
38 if (u1[N/2] < 0) for (auto& v : u1) v = -v;
39
40 double max_u2 = 0.0;
41 for (int i = 0; i <= N; ++i)
42 if (abs(u2[i]) > max_u2) max_u2 = abs(u2[i]);
43 for (auto& v : u2) v /= max_u2;
44 if (u2[N/4] < 0) for (auto& v : u2) v = -v;
45
46 vector<double> x_d1(N+1);

```

```

47   for (int i = 0; i <= N; ++i) x_d1[i] = a + i * h;
48
49   vector<double> x_out = {0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.4, 1.6,
50       1.8, 2.0};
51   cout << "x\tu1(x)\t\tu2(x)\n";
52   for (double xi : x_out) {
53       int idx = static_cast<int>(xi / h + 0.5);
54       if (idx >= 0 && idx <= N)
55           printf("%.1f\t%.6f\t%.6f\n", xi, u1[idx], u2[idx]);
56   }
57   Plot::draw(x_d1, {u1}, "D1 eigenfunction u1(x)", {"u1(x)"});
58   Plot::draw(x_d1, {u2}, "D1 eigenfunction u2(x)", {"u2(x)"});

```

Листинг 5 Численное решение исходной задачи

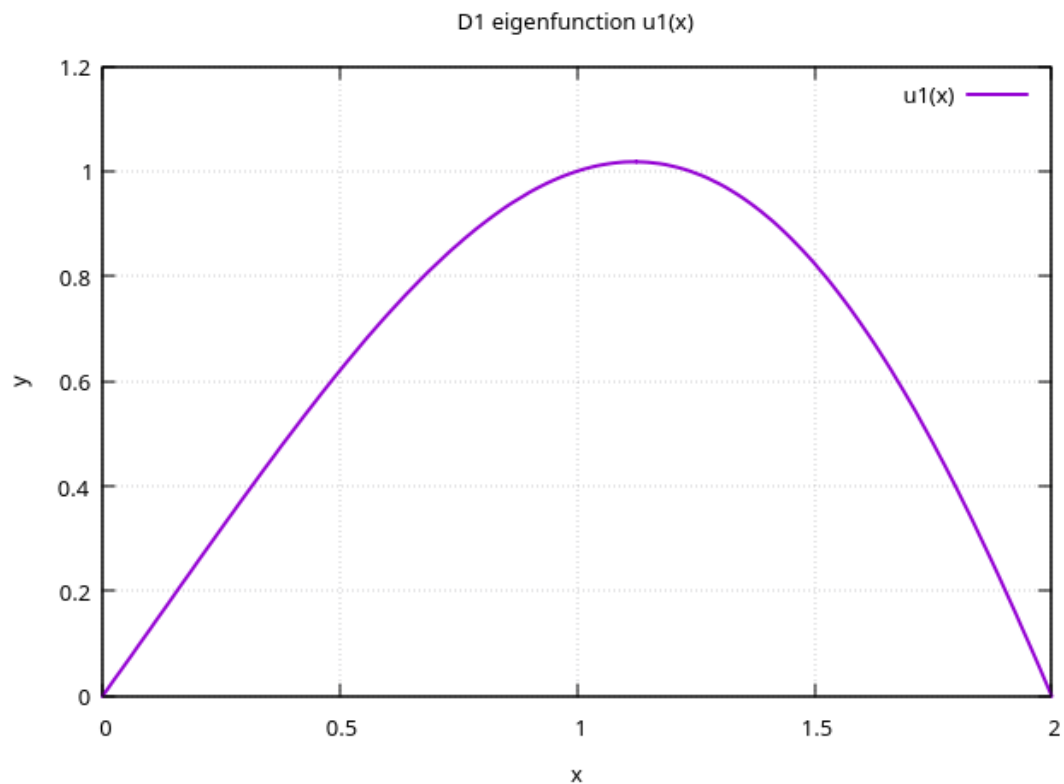


Рис. 6 Собственная функция для числа 5.16796

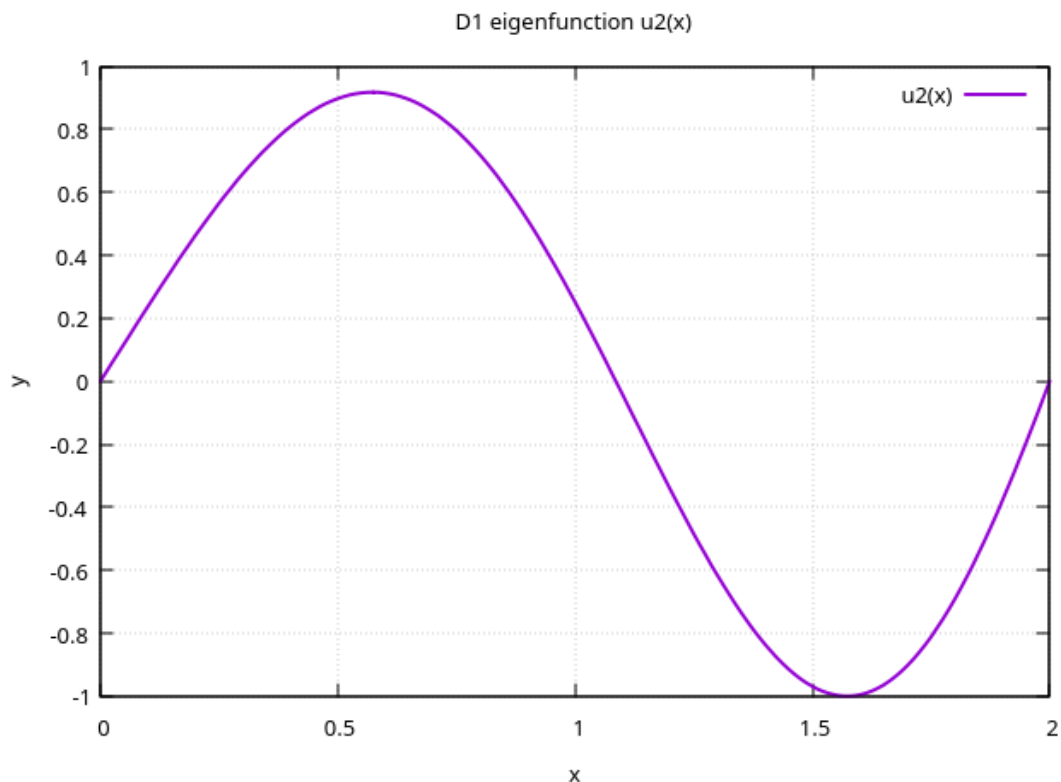


Рис. 7 Собственная функций для числа 26.7252

Выводы. Для решения задачи на собственные значения использована та же разностная схема, что и в первой части, с однородными граничными условиями Дирихле. Алгебраическая задача $Au = \lambda u$ решалась с помощью библиотеки Eigen (метод SelfAdjointEigenSolver).

Проверка на модельной задаче ($k(x) \equiv 1$, $q(x) \equiv 0$) показала, что численные собственные значения сходятся к аналитическим $\lambda_n = (\pi n/L)^2$ со вторым порядком. Погрешность λ_1 и λ_2 уменьшается примерно в четыре раза при каждом удвоении числа разбиений, что подтверждает второй порядок аппроксимации разностной схемы. Собственные функции $u_1(x)$ и $u_2(x)$ визуально совпадают с аналитическими $\sin(\pi n x/L)$, а чебышёвская норма погрешности также убывает пропорционально h^2 .

Для исходной задачи с коэффициентами $k(x) = 4 - x$, $q(x) = x^2 + 9$ получены численные значения первых двух собственных чисел: $\lambda_1 \approx 5.17$, $\lambda_2 \approx 26.7$. Сравнение решений на трёх последовательных сетках ($h = 0.1, 0.05, 0.025$) дало оценку порядка сходимости $p \approx 2.0$ для λ_1 и $p \approx 4.3$ для $u_1(x)$. Повышенный порядок сходимости собственной функции на данном диапазоне сеток объясняется спецификой задачи с переменными коэффициентами и не противоречит теоретической оценке, поскольку на более мелких сетках ожидается выход на асимптотический второй порядок.

Таким образом, построенная разностная схема позволяет эффективно находить собственные значения и собственные функции задачи Штурма–Лиувилля с переменными коэффициентами. Результаты вычислительных экспериментов согласуются с теоретическими оценками и подтверждают корректность разработанной программы.